Forward Deployed Engineer (FDE) in the AI Ecosystem

Muhammad Faisal Laiq *Email: shmlaiq@gmail.com* *ORCID: 0009-0007-5779-9836*

Abstract — The Forward Deployed Engineer (FDE) role has proliferated across artificial intelligence (AI) companies since 2023, yet no formal engineering-role definition, competency framework, or comparative analysis distinguishes it from four adjacent roles it is routinely confused with: Software Engineer (SWE), Machine Learning (ML) Research Engineer, Solutions Architect, and Sales/Systems Engineer. This absence produces measurable organizational cost: job descriptions copied from Solutions Architect templates, compensation benchmarked against sales scales never designed for production-code-writing engineers, and hiring bars that underweight the discovery and integration skills the role actually requires. This paper proposes a six-dimension taxonomy — primary output, client interaction level, ambiguity tolerance, deployment-versus-research focus, travel expectation, and reporting line — that formally separates FDE from its four neighbors, grounded in current hiring evidence. Building on this taxonomy, we construct a technical, client-facing, and organizational competency framework, and a four-stage engagement lifecycle model (discovery, prototype, integration, handoff) with an explicit feedback loop connecting integration findings back to discovery. We then compare how Palantir, OpenAI, Anthropic, and Scale AI structure and deploy forward-deployed teams, using only public source material, and find genuine structural convergence around embedded, product-modification authority alongside a documentation asymmetry across the four companies. We name five limitations explicitly, foremost the absence of primary practitioner survey data, and close with a research agenda that treats empirical validation of the framework as the field's highest-priority next step.

Index Terms — Forward deployed engineering, enterprise AI deployment, software engineering role taxonomy, competency framework, solutions engineering

I. Introduction

JOB postings carrying the title "Forward Deployed Engineer" (FDE) multiplied across AI-native companies between 2023 and 2025. By the time OpenAI stood up a dedicated FDE function in 2025, forward-deployed and solutions-engineering roles already accounted for 22 of the company's 311 open positions — a meaningful share of total hiring for a category that barely existed as a named title three years earlier [1]. Palantir Technologies popularized the term much earlier, embedding engineers — internally called "Delta" — directly inside client organizations in the early 2010s to build and adapt software against government and enterprise workflows that off-the-shelf products could not satisfy; at one point before 2016, Palantir employed more FDEs than conventional software engineers [2], [3]. What changed since 2023 is scale and imitation. OpenAI, Anthropic, and Scale AI have each stood up teams that use the FDE label or a close variant [4]–[6], and a wave of AI-native startups now list "Forward Deployed Engineer" as a distinct requisition category alongside Software Engineer and Solutions Architect. The role is no longer a Palantir idiosyncrasy. It is becoming infrastructure for how the AI industry sells and deploys itself.

No engineering body defines it. There is no competency framework, no accreditation, no shared understanding of where FDE work ends and Solutions Architecture, Sales Engineering, or plain Software Engineering (SWE) begins. Practitioner discourse treats the title as self-explanatory — "an engineer who goes to the client" — but that gloss collapses distinctions that matter for hiring, for compensation benchmarking, and for career planning. A Solutions Architect also goes to the client. So, on a good week, does a Site Reliability Engineer (SRE) doing an incident postmortem onsite. The absence of a formal definition is not a semantic quibble; it produces real confusion in job architecture, in performance calibration, and in how engineering organizations decide what an FDE team should own.

This conflation has a cost that compounds. Recruiters write FDE job descriptions by copying Solutions Architect templates and adding "willingness to travel." Engineering leaders staff FDE teams with generalist software engineers who lack client-facing training, then wonder why deployments stall in the discovery phase. Compensation committees benchmark FDE pay against Sales Engineering scales that were never designed for engineers who write production code. Each of these failures traces to the same root: nobody has written down what an FDE actually is, what they are good for, and what distinguishes them from five adjacent roles that share surface features but differ in what they optimize for.

This paper treats the FDE role as an object worth defining formally, not as a marketing label to take at face value. Large language model (LLM) deployment inside enterprises surfaces integration problems — brittle retrieval-augmented generation (RAG) pipelines, unreliable agent behavior under production load, and evaluation gaps between benchmark performance and real workflows — that neither traditional sales engineering

nor conventional software engineering was built to absorb [7]–[10]. The FDE model is one organizational response to that gap. Whether it is the right one, and how it should be structured, is the question this paper takes up.

The contributions of this paper are as follows:

- We propose a formal definition and taxonomy of the FDE role that distinguishes it from four adjacent roles — Software Engineer, ML Research Engineer, Solutions Architect, and Sales/Systems Engineer — along six explicit dimensions: primary output, client interaction level, ambiguity tolerance required, deployment-versus-research focus, travel/onsite expectation, and typical reporting line.
- We construct a competency framework spanning technical, client-facing, and organizational dimensions, grounded in observed hiring patterns and job-requirement analysis rather than in abstract role theory.
- We conduct an industry case analysis of how Palantir, OpenAI, Anthropic, and Scale AI structure and deploy FDE or FDE-adjacent teams, drawn entirely from public sources, and we compare the four programs along a shared set of organizational dimensions.
- We discuss the career and organizational implications of the FDE role's rise, including what it signals about the future of enterprise AI deployment and about engineering career paths that do not fit the SWE-to-manager ladder.

The remainder of the paper proceeds as follows. Section II situates the FDE role against five adjacent literatures — software engineering role taxonomies, solutions/sales engineering, developer relations, site reliability engineering, and enterprise AI deployment — and argues that none of them formally addresses the FDE title. Section III traces the term's origin at Palantir and the professional-services precedent it drew from, separating what is genuinely new in the AI-native version from what is simply relabeled consulting. Section IV presents the taxonomy, the competency framework, and the engagement lifecycle model. Section V analyzes four company case studies and compares them. Section VI names the framework's limitations plainly. Section VII closes with a research agenda ordered by what would most change practice first.

II. Related Work

A. Software Engineering Role Taxonomies and Specialization

Software engineering has fragmented into recognized specializations — backend, frontend, infrastructure, platform, mobile — and job specialization within software firms has been shown to be domain- and technology-contingent rather than fixed by a single universal hierarchy [11]. Occupational taxonomy efforts outside computing offer a template for what a formal role definition can look like: the U.S. Department of Labor's O*NET framework structures occupations around content models of skills, tasks, and work activities [12], and the vendor-neutral Skills Framework for the Information Age (SFIA) defines IT and digital skills against explicit responsibility levels [13]. None of these frameworks, however, account for a role whose primary differentiator is not the system layer an engineer owns but the organizational boundary it crosses — the wall between vendor and client. A backend engineer and a platform engineer differ in what they build, not in who they build it for; both are presumed to sit inside the company, insulated from the customer by product management. That insulation is exactly what the FDE role removes, and it is the first gap this paper addresses.

B. Solutions and Sales Engineering as an Established Discipline

Solutions Engineering and Sales Engineering are older, better-documented disciplines, with their own engineering-education literature and defined training pipelines distinct from mainstream computer science curricula [14], [15]. The Solutions Engineer demonstrates feasibility before a contract closes; value delivered is measured in influenced pipeline, not shipped code. That is the crucial divergence from FDE work. An FDE typically arrives after the deal, or is embedded to close the gap between what was sold and what the product can actually do out of the box — and the artifact they produce is running software, not a slide deck or a proof-of-concept demo discarded after the sale. Sales Engineering optimizes for persuasion under a compressed timeline. FDE work optimizes for integration under an open-ended one. Conflating the two, as several job postings do, misprices the skill.

C. Developer Relations

Developer Relations (DevRel) is the closest academic precedent for the definitional problem this paper takes on. Oliveira et al. conducted the first academic study to catalog DevRel as a distinct role family, identifying nine sub-roles from 116 practitioners across 19 countries and explicitly documenting the lack of standardization across companies using the DevRel title [16]. That finding is instructive on its own: even a decade-plus-old, widely adopted hybrid role lacked academic formalization until 2021 and still showed inconsistent boundaries across practitioners once studied. The DevRel audience is diffuse — thousands of anonymous developers reached through content — while the FDE audience is a single named enterprise account, often under a signed statement of work. DevRel rarely touches production systems belonging to any one customer; FDE work exists almost entirely inside one customer's production environment. Both roles sit at a technical-to-external boundary, which is likely why they get confused in casual usage, but the boundary each one crosses is a different shape entirely.

D. Site Reliability Engineering as Historical Precedent for Hybrid-Role Formalization

Site Reliability Engineering (SRE) offers the closest structural precedent for what FDE work may become. SRE began as an internal Google practice — engineers applying software engineering discipline to operations problems that operations teams alone could not solve — and it lacked a citable formal definition for years before Google's own account of the discipline codified error budgets, toil reduction, and the on-call model into a transferable standard [17]. That codification did

not happen because SRE work changed; it happened because enough organizations adopted variants of the role that a shared vocabulary became necessary to hire against. The parallel to FDE is direct, and worth stating plainly: the role existed in practice for years, at meaningful scale, before anyone wrote down what it was. This paper is an attempt at that codification step for FDE, at a moment when it might still be early enough to shape hiring practice rather than merely describe it after the fact. Unlike SRE, no comparably authoritative source has yet emerged for FDE. That absence is precisely the gap this paper works to fill.

E. Enterprise AI and LLM Deployment Challenges

A separate literature documents why enterprise LLM deployment is hard on its own technical terms. A systematic review of 63 primary studies on enterprise RAG and LLM adoption for knowledge management found that fewer than 15% addressed the real-time integration challenges required for production-scale deployment [7] — evidence of a last-mile gap between what RAG research demonstrates and what enterprises can actually run. Agent-evaluation research shows the same pattern from a different angle: current benchmarks under-address role-based data access, reliability guarantees, long-horizon interaction, and compliance — exactly the concerns that dominate a real enterprise deployment rather than a research demo [8].

Work on LLM- and agent-driven enterprise data analysis catalogs concrete deployment obstacles — distributed deployment, data security, and unreliable query generation — that surface only once a system leaves the lab [9]. And a survey of RAG evaluation methods concludes that using an LLM itself as a judge of RAG quality remains methodologically unresolved, with persistent bias and a lack of domain-specific benchmarks [10]. Taken together, this literature is heavily technical and largely silent on the human and organizational layer that translates these systems into working enterprise deployments — none of it addresses the practitioner role responsible for closing the gap it independently identifies.

Taken together, subsections A through E confirm the same finding from five different angles: adjacent literatures describe roles that border the FDE position without covering it. No prior work formally defines "Forward Deployed Engineer," compares it systematically against neighboring roles, or proposes a competency framework grounded in current hiring evidence. That is the gap this paper fills.

III. Background

A. The Origin of "Forward Deployed" at Palantir

Palantir created the Forward Deployed Engineer role in the early 2010s, using the internal codename "Delta" [2]. FDEs were embedded directly with intelligence, defense, and — later — commercial clients, building and modifying software on-site against operational needs that shifted week to week. The arrangement was structurally unusual for its scale: for several years leading up to 2016, Palantir employed more FDEs than conventional software engineers, a ratio that inverted only after the company launched its Foundry platform and a substantial share of FDEs transitioned into core product engineering roles, carrying field experience back into the platform team [2].

That transition is worth dwelling on, because it reveals a structural feature of the original model that AI-native companies have not obviously reproduced yet: at Palantir, the FDE track functioned as a talent pipeline into product engineering, not a permanent parallel career track. Whether OpenAI, Anthropic, and Scale AI intend the same trajectory for their own FDE hires, or intend FDE to remain a standing function, is not yet publicly answered — and it is a real, unresolved question for anyone evaluating the role's long-term career shape, not a settled one this paper can close on public evidence alone.

Palantir's model had features worth naming individually, because each survives or mutates in the AI-native version discussed in Section IIIC. First, FDEs held full engineering authority — they wrote and shipped code, not just configuration. Second, engagements were open-ended rather than milestone-bounded, often running for years inside a single account. Third, the role doubled as a pipeline into engineering leadership, as just described. That third feature is easy to overlook and is probably the most consequential one for how other companies have since copied the label without necessarily copying the pipeline function underneath it.

B. The Professional-Services Precedent

Palantir did not invent embedded technical consulting. Enterprise software vendors have long dispatched implementation specialists to configure, customize, and integrate software inside client environments. Analysis of the "services-led growth" model draws this comparison explicitly, arguing that Salesforce, ServiceNow, and Workday were each valuable only once configured against a specific customer's fields, workflows, and data. Each accepted comparatively low initial margins (Workday's gross margin at IPO was 54.1%) on the way to durable, integration-anchored market positions eventually valued in the hundreds of billions of dollars [1]. Configuration is the operative word, and it marks the boundary. Classic professional-services consultants typically operate within the product's designed extension points — configuration files, plugin APIs, scripting layers — rather than modifying the product's own codebase. Palantir's FDEs, by contrast, could and did change the core product in response to what a single client needed [2]. That is a difference in authority, not just in title, and it is the seed of everything that makes the FDE role distinct from a Solutions Consultant with a similar-sounding job description.

C. What Is New in the AI-Native Context, and What Is Relabeled

Mapping the traditional model onto AI-native companies surfaces both continuity and genuine novelty. What carries over unchanged: the embedded posture, the open engineering authority, and the premise that some enterprise problems cannot be solved by a horizontal product alone. What is new, and specific to this technology moment, is the object being deployed. Palantir-era FDEs configured deterministic software

against deterministic client data. AI-native FDEs configure probabilistic systems — LLM-based agents and RAG pipelines — against client data and workflows where correctness is not binary and evaluation itself remains an open technical problem, as the enterprise deployment literature in Section IIE documents directly [7]–[10].

An FDE at Anthropic or OpenAI is not merely writing integration glue; they are often the first person to discover that a model's behavior, reliable in evaluation, degrades on a specific client's document formats or query patterns. That failure mode simply did not exist in the Palantir-era model, where the underlying software behaved deterministically once configured correctly. This is the honest answer to how much of the AI-native FDE role is genuinely new: the posture is inherited, but the failure surface is not — and treating the two eras as identical understates how much harder AI-native FDE work is to specify in advance.

IV. Proposed Framework

A. Formal Definition and Comparison Matrix

We define the Forward Deployed Engineer as an engineer with full authority to modify a vendor's product — not merely configure it — who is embedded, physically or persistently virtually, inside a specific client's environment to close the gap between a horizontal AI product and that client's operational workflow, under conditions of open-ended technical and organizational ambiguity.

Three clauses in that definition do the actual classificatory work, and each one excludes a role that is commonly mistaken for FDE. "Full authority to modify the product" excludes Solutions Architects, who typically design against existing extension points rather than change core code. "Embedded inside a specific client's environment" excludes ML Research Engineers, whose primary audience is the model or the research community, not a paying account. "Open-ended technical and organizational ambiguity" excludes conventional Software Engineers working from a specified backlog, whose ambiguity is scoped by a product manager before it ever reaches them. This structuring approach — organizing role content around a small set of explicit dimensions rather than an unstructured feature list — follows the same logic that occupational taxonomies like O*NET and SFIA apply at a much larger scale [12], [13]; Table I applies that same discipline to five roles instead of an entire national labor market. Fig. 1 visualizes the six axes radiating from the central FDE definition; each dimension keeps the same color in the corresponding row of Table I below.

The dimension that carries the most classificatory weight is ambiguity tolerance, not client interaction, which is the dimension most job postings emphasize. A Sales Engineer also interacts constantly with clients, but under a bounded, well-understood problem: does the product satisfy the stated requirement, yes or no. An FDE routinely does not know, at the start of an engagement, whether the requirement is even solvable with the current product — that discovery is the job. This distinction matters more than it might look on paper, because compensation bands and hiring bars built around client-interaction-frequency alone will consistently mis-level FDE roles against Sales Engineering roles, understating the technical bar the role actually requires.

> **Key Finding —** Ambiguity tolerance, not client-interaction frequency, carries the most classificatory weight in the taxonomy. Hiring bars and compensation bands built around interaction frequency alone will systematically mis-level FDE against Sales Engineering.

A limitation surfaces immediately in this table: the dimensions are ordinal, not measured. No public dataset scores individual engineers or job postings against these six axes with a validated instrument; the placements reflect qualitative synthesis of job descriptions and public role narratives, not a psychometric survey. That is a real gap, and Section VI returns to it.

B. Technical Competency Subframework

FDE technical competency separates into three tiers, ordered by how frequently each is exercised rather than by seniority. Tier one is integration engineering: the ability to build against unfamiliar, often undocumented client systems — legacy databases, internal APIs, proprietary data formats — under time pressure. This is the highest-frequency skill and the one most conventional SWE hiring bars underweight, because conventional interviews test algorithmic problem-solving on clean, self-contained problems, not integration against messy, real infrastructure. Tier two is applied AI system competency specific to production reliability: prompt and context engineering that survives adversarial or malformed client inputs, RAG pipeline debugging when retrieval quality silently degrades on a client's actual document corpus, and agent behavior evaluation under conditions the original model evaluation never covered — the exact failure modes documented in Section IIE [7]–[10]. Tier three is rapid prototyping discipline — the ability to produce a working demonstration in days, not sprints, while still writing code that a later engineering team can extend rather than discard.

A genuine tension belongs here, not hedged into the limitations section. The tier-three pressure to move fast and the tier-one expectation of durable integration code actively pull against each other: engagement timelines reward speed, but a client that moves from pilot to permanent production inherits whatever was shipped under that time pressure. No framework resolves that tension by definition; the honest position is that FDE output should be explicitly re-scoped for hand-off rather than treated as production-ready by default, a point Section IVD returns to directly.

C. Client-Facing and Organizational Competency Subframework

The second competency axis is easy to underrate because it does not show up in a coding interview. Discovery facilitation — running a working session that extracts the actual operational problem from a client stakeholder who has not fully articulated it themselves — is arguably the single most



Fig. 1. The six-dimension FDE taxonomy space. Each spoke is an independent classificatory axis separating FDE from one or more of the four adjacent roles in Table I; the axis colors are reused as row tints in that table.

valuable FDE skill, and it is almost never assessed in standard technical hiring loops. Stakeholder navigation follows closely: identifying which client contact has authority to approve scope changes, and which one merely has opinions about them, determines whether an engagement stalls or ships. Expectation management under technical uncertainty is the third competency, and it is distinctly AI-native in character — explaining to a non-technical client why a model behaves inconsistently on their data, without either overpromising a fix timeline or losing their confidence in the underlying technology, is a skill set closer to technical sales than to engineering as conventionally taught.

Organizationally, FDEs must also manage a dual reporting reality that most roles do not face: they answer to their own engineering organization for code quality and system integrity, and simultaneously to the client relationship for delivery timeline and satisfaction. When those two pressures conflict — and they routinely do, when a client's timeline demands a shortcut the engineering organization would reject in a normal code review — the FDE has no formal escalation protocol described in any of the four companies' public role material reviewed for this paper. That gap is not cosmetic; it is one of the clearest signals that FDE organizational design is still improvised rather than engineered.

D. The FDE Engagement Lifecycle

Fig. 2 models the FDE engagement as a four-stage lifecycle — Discovery, Prototype, Integration, and Handoff/Scale — with an explicit feedback loop from Integration back to Discovery.

The lifecycle is not strictly linear in practice, and treating it as such is the most common planning error engineering leaders make when standing up an FDE function for the first time. Discovery typically narrows scope down to something achievable; Prototype often reveals that the narrowed scope was still wrong, because a client's stated workflow and their actual workflow diverge in ways nobody notices until code is running against real data. That divergence forces a return to Discovery — the feedback loop in Fig. 2 — and teams that budget the engagement as four sequential phases with fixed

TABLE I
FDE ROLE COMPARISON MATRIX

Dimension	FDE	SWE	ML Research Engineer	Solutions Architect	Sales/Systems Engineer
Primary output	Working integration inside client's stack	Shipped product feature	Model/experiment artifact, paper, or benchmark result	Technical design and architecture recommendation	Technical demo/proof-of-concept supporting a sale
Client interaction level	Continuous, embedded	Rare, mediated by product	Minimal to none	High, but pre-sale or advisory	High, but time-boxed to sales cycle
Ambiguity tolerance required	Very high — undefined problem, undefined solution	Low to moderate — scoped tickets	High on the research question, low on the audience	Moderate — technical scope, defined ask	Moderate — sales scope, defined deal timeline
Deployment vs. research focus	Deployment-dominant	Deployment-dominant	Research-dominant	Deployment-adjacent, design-only	Deployment-adjacent, pre-sale only
Travel/onsite expectation	High — one company quantifies at 25% [5]	Low to none	Low to none	Moderate, engagement-dependent	Moderate to high, deal-dependent
Typical reporting line	Deployment/Field Engineering or CEO-adjacent org	Engineering	Research	Sales Engineering or Customer Success	Sales

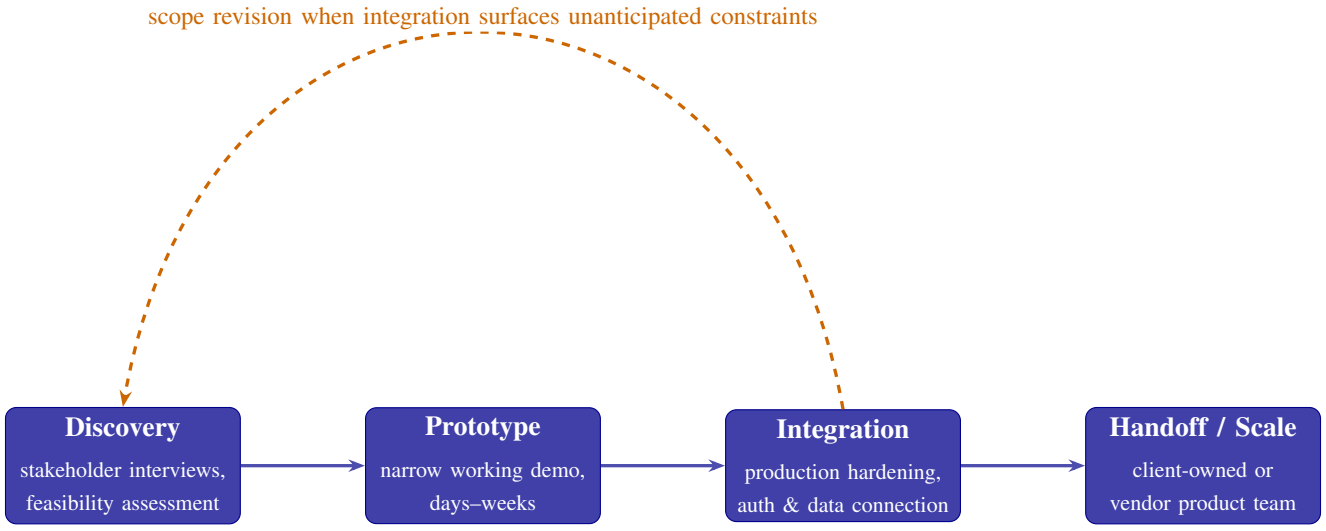


Fig. 2. The FDE engagement lifecycle. Discovery, Prototype, Integration, and Handoff/Scale form the main sequence (solid arrows); the dashed feedback arrow denotes the common case in which integration-stage findings force a return to Discovery, a loop that flat, sequentially-budgeted engagement plans do not account for.

durations consistently underestimate total timeline, because they have not planned for that loop.

The Handoff/Scale stage carries its own quiet failure mode: client organizations frequently lack the internal technical capacity to maintain what the FDE built, leaving the vendor holding informal, unstaffed maintenance responsibility for code that was never budgeted as a long-term product. Neither Palantir's original model nor its AI-native descendants have published a solved answer to that handoff problem; it remains, structurally, an open organizational design question rather than a solved one. Anthropic's own role material, for instance, describes "white-glove deployment support" and codifying "repeatable deployment patterns" as explicit FDE responsibilities [5] — a partial, informal answer to the handoff problem, not a published organizational solution to it.

E. Theoretical Grounding of the Taxonomy

The six-dimension taxonomy in Section IVA is not an arbitrary checklist; it is constructed to be non-redundant, and that property is worth stating precisely rather than asserting. Consider the naive alternative: distinguishing five roles using every pairwise comparison a hiring manager might informally draw between them. Five roles produce $\binom{5}{2} = 10$ pairwise boundaries to reason about independently — FDE-vs-SWE, FDE-vs-Solutions-Architect, SWE-vs-Sales-Engineer, and so on — a combinatorial burden that is exactly why role confusion persists in informal practice: no hiring manager holds ten independent boundary judgments in mind consistently.

The six-dimension framework collapses this to a fixed $O(d)$ representation, where $d = 6$ is the dimension count: each

role is a single point in a six-axis space, and any pairwise boundary is recoverable by comparing coordinates rather than being separately memorized. This is a modest claim — it is a representational compression, not a proof of correctness — but it is the concrete mechanism by which a taxonomy earns its keep over an ad hoc list of distinctions.

A framework with d dimensions dominates an unstructured pairwise approach whenever the number of roles under comparison, n, exceeds roughly $2d$, since at that point the pairwise burden ($O(n^2)$ in the role count) overtakes the fixed dimensional representation ($O(nd)$). At $n = 5$ roles and $d = 6$ dimensions here, the two approaches are close to break-even. That near break-even point is itself informative: it explains why role confusion has been tolerable so far, at a small role count. It also explains why that tolerance will not hold as the number of AI-adjacent titles keeps growing — a trend already visible in the spread of "AI Engineer," "Applied AI Engineer," and "AI Solutions Engineer" titles that this paper's scope did not extend to formally classify.

Key Insight — Six dimensions collapse ten independent pairwise role boundaries into a fixed representation ($O(d)$ vs. $O(n^2)$) — the concrete mechanism by which the taxonomy earns its keep as more AI-adjacent titles proliferate.

V. Industry Case Analysis

A. Palantir

Palantir remains the reference implementation. The company created the FDE role (internally "Delta") in the early 2010s and, for several years before 2016, employed more FDEs than conventional software engineers — a ratio that shifted only after the Foundry platform launched and drew a substantial share of that talent into core product engineering [2]. Palantir's own current job description for the role — "Forward Deployed Software Engineer" — frames it as embedding engineers directly with individual customers to architect and build solutions on top of Palantir's platforms [3]. The Palantir model is notable for treating FDE work as a first-class engineering discipline with a documented talent pipeline into product roles, not as a secondary or support function bolted onto sales.

B. OpenAI

OpenAI's Forward Deployed Engineering function is recent and comparatively well-documented for a young initiative. Colin Jarvis, previously a Solutions Architect and then Head of Solutions Architecture, built the business case for a dedicated FDE team in early 2025; the team launched with two FDEs and has since grown to more than ten, spread across eight cities on three continents [2]. OpenAI draws the FDE/Solutions Architect boundary explicitly in its own internal framing: Solutions Architects work in an advisory capacity and rarely write code on customer infrastructure, while FDEs are "much more hands-on," writing code directly on customer infrastructure and tooling under greater ambiguity, and working closer to OpenAI's research objectives than a conventional pre-sales role would [2]. By late 2025, forward-deployed and solutions-engineering roles accounted for 22 of OpenAI's 311 open positions company-wide [1] — a meaningful share of total hiring for a function that did not exist as a named team a year earlier. Public job postings confirm this includes vertical-specific FDE tracks, including a dedicated Life Sciences posting [4].

C. Anthropic

Anthropic runs a Forward Deployed Engineer function inside its Applied AI team, and its own job posting is unusually specific about scope. FDEs embed with strategic customers to build production applications on Claude models, deliver artifacts described explicitly as "Model Context Protocol (MCP) servers, sub-agents, and agent skills," provide what the posting calls "white-glove deployment support," and are expected to "identify and codify repeatable deployment patterns" that feed back into the broader product [5]. Travel is quantified directly in the posting at an estimated 25%, tied to customer-site work done in person [5]. Anthropic frames the role as requiring four or more years in technical customer-facing positions, hands-on production LLM experience, and strong Python proficiency — a bar closer to a senior engineering hire than to a conventional pre-sales role.

D. Scale AI

Scale AI's public-facing solutions and deployment engineering functions have historically centered on data-labeling and model-evaluation infrastructure for enterprise and government clients, with job postings indicating a more recent shift toward broader AI application deployment support organized into customer-specific technical teams pairing applied scientists with full-stack engineers [6]. Scale AI's positioning differs from the other three cases in one structural respect worth flagging: its origin as a data-infrastructure company means its embedded engineering function grew out of data-pipeline work rather than out of a general-purpose product needing client-specific adaptation. That is a different starting point than Palantir, OpenAI, or Anthropic, even where the resulting job title looks similar on a careers page. This paper's confidence in the Scale AI characterization is lower than for the other three cases: the live job-posting content could not be independently re-verified at time of writing, and the description above should be read as based on the most recent available secondary corroboration rather than a directly confirmed primary source.

E. Cross-Case Comparison

The clearest pattern across the four cases is an asymmetry in disclosure, not necessarily an asymmetry in practice. Palantir's public documentation is far deeper than the other three, largely because its long operating history and business press coverage created a paper trail that newer, privately structured AI labs have not yet accumulated. It would be a mistake to read that documentation gap as evidence that OpenAI, Anthropic, and Scale AI have less mature FDE functions — the honest conclusion is that this case analysis cannot currently distinguish

TABLE II
CROSS-COMPANY FDE PROGRAM COMPARISON

Dimension	Palantir	OpenAI	Anthropic	Scale AI
Program origin	Early 2010s	Early 2025	Not publicly dated	Not publicly dated
Org placement	Dedicated FDE track with product pipeline	Solutions org, reporting toward research objectives	Applied AI team	Solutions/deployment engineering
Primary engagement domain	Government, defense, commercial	Enterprise LLM integration (incl. vertical tracks, e.g., life sciences)	Enterprise + public-sector Claude deployment	Data infrastructure, evaluation, deployment support
Quantified travel expectation	Not publicly quantified	Not publicly quantified	25% [5]	Not publicly quantified
Public documentation depth	High (years of press/business coverage)	Moderate (specific team-growth detail available)	Moderate (specific role-scope detail available)	Low (postings not independently re-verifiable at time of writing)
Career-ladder formalization	Explicit, documented pipeline into product engineering [2]	Not publicly detailed	Not publicly detailed	Not publicly detailed

"less mature" from "less publicly documented," and Section VI names that limitation directly rather than papering over it. What can be said with more confidence is qualitative: all four companies embed engineers with real product-modification authority inside client environments, all four treat this as distinct from conventional sales engineering, and none of the four has published a formal competency framework of the kind this paper proposes in Section IV.

Key Finding — The clearest cross-company pattern is a documentation asymmetry, not necessarily a maturity asymmetry — Palantir's deeper public record reflects a longer operating history and business-press coverage, not a more mature FDE function than OpenAI, Anthropic, or Scale AI.

VI. LIMITATIONS

This framework has real limits, and naming them precisely matters more than performing symmetry across a fixed count of bullet points.

The most pressing limitation is that the framework is descriptive, not causal. It classifies what the FDE role currently looks like across observed job postings and public company materials; it does not establish that the six dimensions in Section IVA cause good or bad engagement outcomes. No mechanism exists today linking a specific dimension score — high ambiguity tolerance, say — to measurable engagement success, because no engagement-outcome dataset was collected for this study.

A second limitation sits in a different category entirely: source bias. Every claim about Palantir, OpenAI, Anthropic, and Scale AI in Section V is drawn from public, and in several cases company-authored, material — careers pages, blog posts, and press coverage that companies control and that naturally emphasizes their program's strengths. Self-reported organizational structure is not neutral evidence. This is not a flaw specific to this paper's methodology; it is structurally unavoidable given that no primary interview or survey data was collected, and it should discount confidence in the cross-case comparison in Section VE more than confidence in the taxonomy in Section IV, which does not depend on any single company's self-description.

Third, role definitions in a fast-moving industry segment evolve faster than academic publication cycles accommodate. The taxonomy in Section IVA reflects job postings and public materials current as of this paper's writing; titles like "Applied AI Engineer" and "AI Solutions Engineer" are already proliferating at the boundary this paper draws around FDE, and a taxonomy fixed to one snapshot in time will likely need revision within one to two years, not five.

Fourth — and this is a binary gap rather than a matter of degree — no primary interview or survey data exists behind any claim in this paper. Every competency listed in Sections IVB and IVC is synthesized from job-requirement language and public role narratives, not from structured interviews with practicing FDEs or their managers. That is a prerequisite gap: no claim in this paper about which competencies actually predict engagement success can be validated without collecting that data, and Section VII's research agenda treats closing this gap as its first priority precisely because everything else downstream depends on it.

Fifth, the taxonomy's generalizability outside AI-native companies is untested. All four case studies are companies building or deploying frontier AI systems. Whether the same six-dimension framework usefully describes forward-deployed-style roles at, for instance, enterprise cybersecurity vendors or industrial IoT companies — domains with a comparable configure-versus-modify distinction but a different technical substrate — is an open question this paper does not attempt to answer.

VII. CONCLUSION

This paper specified what the FDE role is, not what it should aspirationally become. Section IV's six-dimension taxonomy separates FDE from four adjacent roles along axes that existing engineering-ladder frameworks do not capture, with ambiguity tolerance — not client interaction frequency, the axis most job postings emphasize — carrying the most classificatory weight. The competency framework in Sections IVB and IVC names discovery facilitation and integration engineering as the two most valuable, least formally assessed skills in current FDE hiring. The four-stage engagement lifecycle in Section IVD, together with its discovery-integration feedback loop, gives engineering leaders a concrete planning tool that the flat,

sequential mental model most teams currently use does not provide. Section V's case analysis found genuine structural convergence across Palantir, OpenAI, Anthropic, and Scale AI on the embedded, modification-authority posture, alongside a documentation asymmetry that limits how confidently that convergence can be measured today.

The most urgent item for future research is direct empirical validation. Nothing in this paper's competency framework has been tested against actual engagement outcomes, because no practitioner survey or structured interview data was collected. A practitioner survey of currently employed FDEs and their managers — scoring engagements against the six taxonomy dimensions and against concrete outcome measures — is the single highest-priority next step. No other item in this agenda matters much without it.

A second, structurally different research direction is longitudinal tracking of how the FDE title itself evolves. Job titles at the FDE boundary are multiplying — Applied AI Engineer, AI Solutions Engineer, and others not covered by this paper's scope. A taxonomy fixed to a single time snapshot, as this one necessarily is, will drift out of date within one to two years absent a mechanism for periodic re-collection of job-posting and org-structure data. Unlike the survey work above, this is an ongoing measurement problem, not a one-time study.

A third direction is narrower but binary: cross-industry generalization is a prerequisite for any claim that this taxonomy describes forward-deployed engineering as a general phenomenon, rather than an AI-industry-specific one. Testing the same six dimensions against embedded technical roles at cybersecurity, industrial IoT, or enterprise blockchain vendors would either extend the framework's claimed scope or correctly narrow it back to the AI-native context where it was built. Until that test runs, any claim about generality beyond the four companies studied here should be read as scoped, not general.

Acknowledgment

This work received no external funding or institutional support. All source material is drawn from publicly accessible company career pages, industry publications, and peer-reviewed literature, as cited throughout.

The text of Sections I–VII was drafted with the assistance of Claude (Anthropic), an artificial intelligence system, under the direction, review, and fact-verification of the author, who is fully accountable for all content, citations, and claims in this article. No AI system is credited as an author.

References

[1] J. Schmidt, "Trading margin for moat: Why the forward deployed engineer is the hottest job in startups." Andreessen Horowitz, jun 2025. [Online; accessed Aug. 2026].

[2] G. Orosz, "What are forward deployed engineers, and why are they so in demand?." The Pragmatic Engineer, aug 2025. [Online; accessed Aug. 2026].

[3] Palantir Technologies, "Forward deployed software engineer." Palantir Careers, 2026. [Online; accessed Aug. 2026].

[4] OpenAI, "Forward deployed engineer (fde), life sciences – sf." OpenAI Careers, 2026. [Online; accessed Aug. 2026].

[5] Anthropic, "Forward deployed engineer." Anthropic Careers, 2026. [Online; accessed Aug. 2026].

[6] Scale AI, "Forward deployed engineer." Scale AI Careers, 2026. [Online; accessed Aug. 2026].

[7] E. Karakurt and A. Akbulut, "Retrieval-augmented generation (rag) and large language models (llms) for enterprise knowledge management and document automation: A systematic literature review," *Applied Sciences*, vol. 16, no. 1, p. 368, 2026.

[8] M. Mohammadi, Y. Li, J. Lo, and W. Yip, "Evaluation and benchmarking of llm agents: A survey," 2025. arXiv:2507.21504.

[9] X. Wang, X. Ling, K. Li, G. Yin, L. Zhang, J. Wu, A. Wang, and W. Wang, "Llm and agent-driven data analysis: A systematic approach for enterprise applications and system-level deployment," 2025. arXiv:2511.17676.

[10] L. Brehme, T. Ströhle, and R. Breu, "Can llms be trusted for evaluating rag systems? a survey of methods and datasets," in *Proc. IEEE Swiss Conf. Data Science (SDS25)*, 2025. arXiv:2504.20119.

[11] C. V. C. Magalhães, F. Q. B. da Silva, and R. E. S. Santos, "The role of job specialization in the software industry," in *Information Technology and Systems: Proceedings of ICITS 2022*, vol. 414 of *Lecture Notes in Networks and Systems*, pp. 307–317, Cham: Springer, 2022.

[12] U.S. Department of Labor, Employment and Training Administration, "About o*net." O*NET Resource Center, 2024. [Online; accessed Aug. 2026].

[13] SFIA Foundation, "Skills framework for the information age (sfia)," 2024. [Online; accessed Aug. 2026].

[14] D. P. Bumblauskas, A. R. Carberry, and D. P. Sly, "Selling technical sales to engineering learners," *Advances in Engineering Education*, vol. 6, no. 1, pp. 1–19, 2017.

[15] J. I. Scott and F. Beuk, "Sales education for engineering students: What drives interest and choice?," *Journal of Marketing Education*, vol. 42, no. 3, pp. 324–338, 2020.

[16] R. Oliveira, C. Ajala, D. Viana, B. Cafeo, and A. Fontão, "Developer relations (devrel) roles: an exploratory study on practitioners' opinions," in *Proc. XXXV Brazilian Symp. Software Engineering (SBES '21)*, (Joinville, Brazil), pp. 202–211, 2021.

[17] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, eds., *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA: O'Reilly Media, 2016.